

WEEK6: Demonstrate Cache, Session Management for Student Login Application.

Project Structure

```
student_login_project/
├── manage.py
├── student_login_project/      # Project configuration
│   ├── __init__.py
│   ├── settings.py           # Cache + session settings
│   ├── urls.py               # Project-level URL routing
│   ├── asgi.py
│   └── wsgi.py
├── studentlogin/              # App for student login
│   ├── __init__.py
│   ├── admin.py              # Register Student model
│   ├── apps.py
│   ├── models.py             # Student model (username, password)
│   ├── views.py              # Login, logout, dashboard (cache + session + cookie)
│   ├── urls.py               # App-specific routes
│   ├── forms.py              # (Optional) Django form for login
│   ├── middleware.py         # (Optional) custom session/cookie checks
│   └── templates/
│       ├── login.html        # Login form
│       └── dashboard.html    # Dashboard with logout link
└── db.sqlite3                # Default database
```

Step1: Create a environment

Step2: Create Django Project –student_login_project

Step3: Create App-studentlogin

Step4:

Settings.py

Application definition

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
```

```

'django.contrib.messages',
'django.contrib.staticfiles',
'studentlogin',
]

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': ['BASE_DIR /TEMPLATES'],
        .....,
    }

CACHES = {
    "default": {
        "BACKEND": "django.core.cache.backends.locmem.LocMemCache",
        "LOCATION": "student-cache",
    }
}

# Sessions are enabled by default
SESSION_ENGINE = "django.contrib.sessions.backends.db"

```

Step5:

Create a Model-Models.py

```

# models.py
from django.db import models
from django.contrib.auth.hashers import make_password, check_password

class Student(models.Model):
    username = models.CharField(max_length=50, unique=True)
    password = models.CharField(max_length=128) # hashed password

    def save(self, *args, **kwargs):
        self.password = make_password(self.password)
        super().save(*args, **kwargs)

    def check_password(self, raw_password):
        return check_password(raw_password, self.password)

    def __str__(self):
        return self.username

```

Step6: Admin Module

```
#admin.py
from django.contrib import admin
from .models import Student

@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):
    list_display = ("username",)
```

Step7:

```
# views.py
from django.shortcuts import render
from django.http import HttpResponse
from django.core.cache import cache
from .models import Student

def login_view(request):
    if request.method == "POST":
        username = request.POST.get("username")
        password = request.POST.get("password")

        # 1 Check cache first
        student = cache.get(username)
        if not student:
            try:
                student = Student.objects.get(username=username)
                cache.set(username, student, timeout=300) # cache for 5 mins
            except Student.DoesNotExist:
                return HttpResponse("Invalid username")

        # 2 Validate password
        if student.check_password(password):
            # 3 Create session
            request.session["student_id"] = student.id
            request.session["username"] = student.username

            # 4 Create cookie
            response = HttpResponse(f"Welcome {student.username}!")
            response.set_cookie("student_username", student.username,
max_age=3600)
            return response
        else:
            return HttpResponse("Invalid password")
```

```

        return render(request, "login.html")

#dashboard
def dashboard_view(request):
    if "student_id" in request.session:
        return HttpResponse(f"Welcome {request.session['username']}! <a
href='/logout/'>Logout</a>")
    else:
        return HttpResponse("Please login first.")

#logout
def logout_view(request):
    # Clear session
    request.session.flush()

    # Clear cookie
    response = HttpResponse("Logged out successfully!")
    response.delete_cookie("student_username")
    return response

Step8:
AppLevel
# studentlogin/urls.py(AppLevel)

from django.urls import path
from . import views

urlpatterns = [
    path('login/', views.login_view, name='login'),
    path('logout/', views.logout_view, name='logout'),
    path('dashboard/', views.dashboard_view, name='dashboard'), # optional
]

# student_login_project/urls.py(project level)
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path("", include('studentlogin.urls')), # include app routes
]

```

Step9:

Create Templates folder (studentlogin/templates)

Login.html

```
<form method="post">
  {% csrf_token %}
  <label>Username:</label>
  <input type="text" name="username" required>
  <label>Password:</label>
  <input type="password" name="password" required>
  <button type="submit">Login</button>
</form>
```

Dashboard.html

```
<h2>Welcome {{ request.session.username }}</h2>
<p>You are logged in as student ID: {{ request.session.student_id }}</p>

<!-- Logout link -->
<a href="{% url 'logout' %}">Logout</a>
```

Commands:

Python manage.py makemigrations

Python manage.py migrate

#Create superuser

python manage.py createsuperuser

Username: admin

Email address: admin@example.com

Password: *****

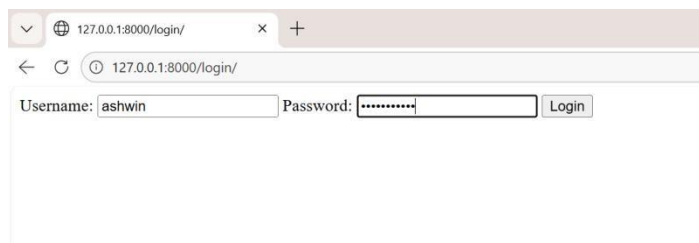
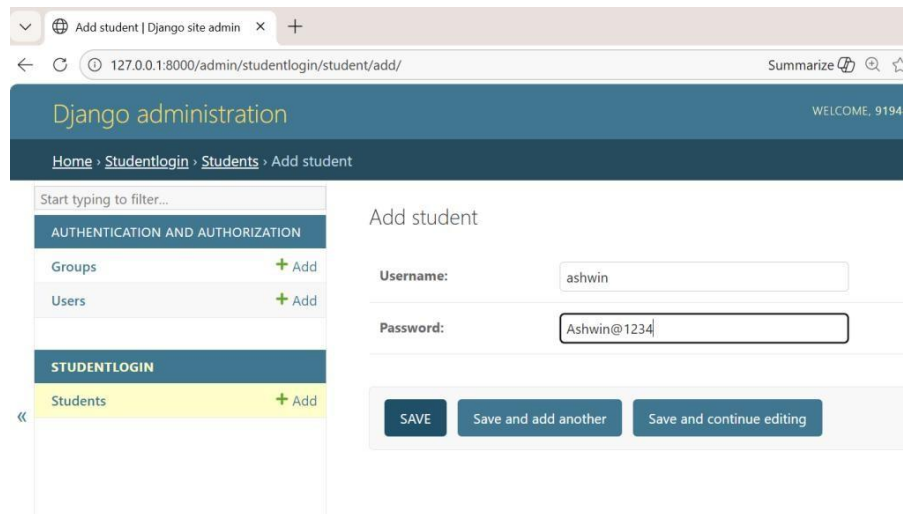
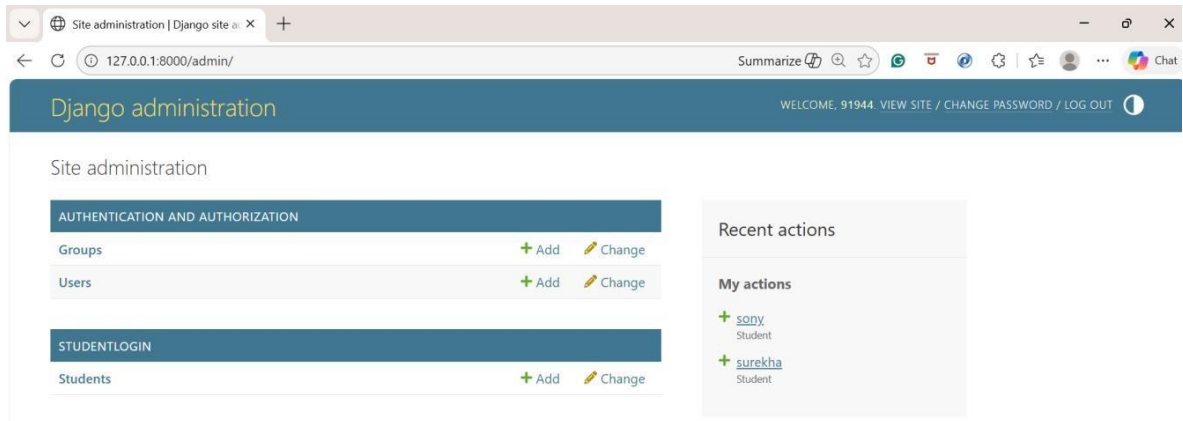
Password (again): *****

Superuser created successfully.

python manage.py runserver

Visit:

http://127.0.0.1:8000/admin/



127.0.0.1:8000/dashboard/

127.0.0.1:8000/dashboard/

Welcome ashwin! [Logout](#)

Application

- Manifest
- Service workers
- Storage

Storage

- Local storage
- Session storage
- Extension storage
- IndexedDB
- Cookies
 - http://127.0.0.1:8000
- Private state tokens

Application

Filter

Name	Value	D...	Pa...	Ex...
csrftoken	U0duV10Mw87...	1...	/	2...
sessionid	2ii281m4wvpkd...	1...	/	2...
student_username	ashwin	1...	/	2...

Cookie Value ☐ Show URL-decoded

ashwin

127.0.0.1:8000/logout/

127.0.0.1:8000/logout/

Logged out successfully!

Application

- Manifest
- Service workers
- Storage

Storage

- Local storage
- Session storage
- Extension storage
- IndexedDB
- Cookies
 - http://127.0.0.1:8000
- Private state tokens

Application

Filter

Name	Value	D...	Pa...
csrftoken	U0duV10Mw87...	1...	/

WEEK7: Create a Quiz Application using Django Framework along with SQLite3.

Steps:

#Create an Environment

```
python -m venv week7
```

```
cd week7/scripts/activate
```

Install Django if not already

```
pip install django
```

Create a new Django project

```
django-admin startproject quiz_app
```

```
cd quiz_app
```

Create a quiz application inside the project

```
python manage.py startapp quiz
```

```
quiz_app/                                # Root project folder
|
├── manage.py                            # Django command-line utility
|
├── quiz_app/                            # Project settings & configuration
|   ├── __init__.py
|   ├── settings.py                      # Global settings (DB, installed apps, etc.)
|   ├── urls.py                          # Root URL routing
|   └── asgi.py                          # ASGI entry point
```

```

|   └─ wsgi.py           # WSGI entry point
|
|   └─ quiz/             # Our quiz application
|       └─ __init__.py
|       └─ admin.py      # Register models for Django admin
|       └─ apps.py       # App configuration
|       └─ models.py     # Database models (Question, Choice)
|       └─ views.py      # Business logic (quiz & results)
|       └─ urls.py       # App-specific URL routing
|       └─ tests.py      # Unit tests
|       └─ migrations/   # Auto-generated DB migrations
|           └─ __init__.py
|           └─ templates/quiz/ # HTML templates
|               └─ quiz.html    # Quiz page
|               └─ result.html  # Results page
|
|   └─ db.sqlite3        # SQLite3 database file
└─ static/              # (Optional) static files (CSS, JS, images)

```

```

Settings.py
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'quiz', # our app
]
DEBUG = True # Keep ON during development

```

```

ALLOWED_HOSTS = ['127.0.0.1', 'localhost']
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / "templates"], # global templates folder
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

```

```

STATIC_URL = '/static/'
STATICFILES_DIRS = [BASE_DIR / "static"]

```

```

LOGIN_REDIRECT_URL = '/'
LOGOUT_REDIRECT_URL = '/'

```

```

LANGUAGE_CODE = 'en-in'
TIME_ZONE = 'Asia/Kolkata'
USE_I18N = True
USE_TZ = True
#security
CSRF_COOKIE_SECURE = True
SESSION_COOKIE_SECURE = True
SECURE_BROWSER_XSS_FILTER = True

```

```

#Models.py
from django.db import models

class Question(models.Model):
    text = models.CharField(max_length=255)

    def __str__(self):
        return self.text

class Choice(models.Model):

```

```

    question = models.ForeignKey(Question, on_delete=models.CASCADE,
related_name="choices")
    text = models.CharField(max_length=255)
    is_correct = models.BooleanField(default=False)

    def __str__(self):
        return self.text

```

Migrate database:
python manage.py makemigrations
python manage.py migrate

#admin.py

```

from django.contrib import admin
from .models import Question, Choice

```

```

admin.site.register(Question)
admin.site.register(Choice)

```

#views.py

```

from django.shortcuts import render
from .models import Question, Choice

```

```

def quiz_view(request):
    questions = Question.objects.all()
    return render(request, "quiz/quiz.html", {"questions": questions})

```

```

def result_view(request):
    if request.method == "POST":
        score = 0
        questions = Question.objects.all()
        for question in questions:
            # Each question's input name should be its ID in the template
            selected = request.POST.get(str(question.id))
            if selected:
                try:
                    choice = Choice.objects.get(id=int(selected), question=question)
                    if choice.is_correct:
                        score += 1
                except Choice.DoesNotExist:
                    pass

```

```

    return render(request, "quiz/result.html", {
        "score": score,
        "total": questions.count()
    })
else:
    # If someone visits result_view directly without POST
    return render(request, "quiz/result.html", {
        "score": 0,
        "total": 0
    })

```

```

#quiz/urls.py(app level)
from django.urls import path
from . import views

```

```

urlpatterns = [
    path("", views.quiz_view, name='quiz'),
    path('result/', views.result_view, name='result'),
]

```

```

#quiz_app/urls.py( Project level)
from django.contrib import admin
from django.urls import path, include

```

```

urlpatterns = [
    path('admin/', admin.site.urls),
    path("", include('quiz.urls')),
]

```

Templates- quiz/templates/quiz/quiz.html

```

<form method="post" action="{% url 'result' %}">
    {% csrf_token %}
    {% for question in questions %}
        <h3>{{ question.text }}</h3>
        {% for choice in question.choices.all %}
            <input type="radio" name="{{ question.id }}" value="{{ choice.id }}">
                {{ choice.text }}<br>
        {% endfor %}
    {% endfor %}
    <button type="submit">Submit</button>
</form>

```

Results.html

```

<h2>Your Score: {{ score }} / {{ total }}</h2>

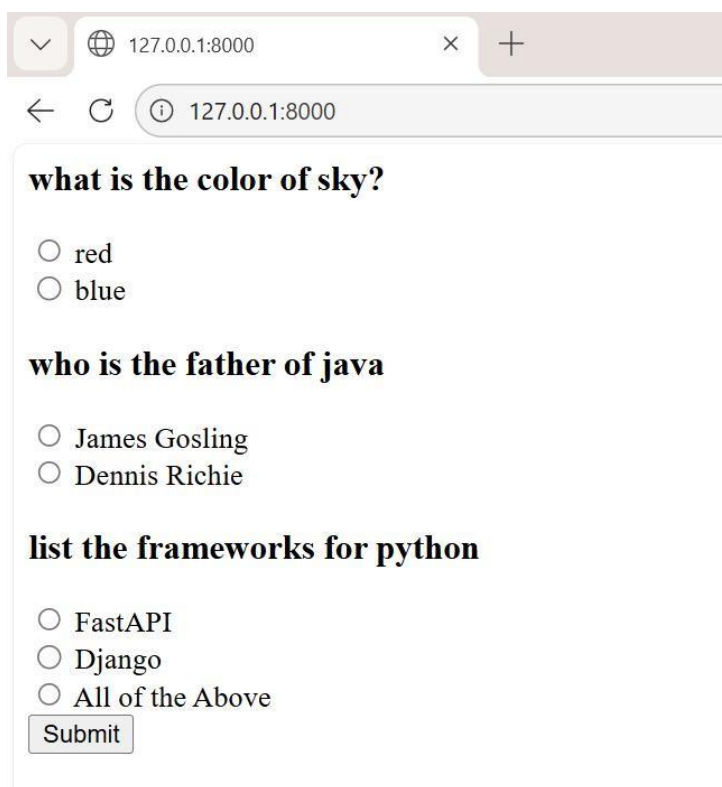
```

Try Again

Run Project:

python manage.py runserver

OUTPUT:



A screenshot of a web browser window. The address bar shows the URL 127.0.0.1:8000. The page content consists of three quiz questions, each with two radio button options. The first question is 'what is the color of sky?' with options 'red' and 'blue'. The second question is 'who is the father of java' with options 'James Gosling' and 'Dennis Richie'. The third question is 'list the frameworks for python' with options 'FastAPI', 'Django', and 'All of the Above'. At the bottom of the form is a 'Submit' button.

127.0.0.1:8000

what is the color of sky?

☐ red

☐ blue

who is the father of java

☐ James Gosling

☐ Dennis Richie

list the frameworks for python

☐ FastAPI

☐ Django

☐ All of the Above

Submit

127.0.0.1:8000

127.0.0.1:8000

what is the color of sky?

☒ red
☐ blue

who is the father of java

☒ James Gosling
☐ Dennis Richie

list the frameworks for python

☐ FastAPI
☒ Django
☐ All of the Above

Submit

127.0.0.1:8000/result/

127.0.0.1:8000/result/

Your Score: 1 / 3

[Try Again](#)